PYTHON BACKEND DEVELOPMENT

Define FastAPI Routes

Comprehensive Guide to Routing, Path Parameters, Query Parameters, APIRouter, and Best Practices

Document Type: Technical Architecture & Developer Guide

Topic: FastAPI Routing & API Design

Target Audience: Backend Engineers, Python Developers, System Architects

Status: Approved Guide

1. Introduction to FastAPI Routes

FastAPI is a modern, fast (high-performance) web framework for building APIs with Python based on standard Python type hints. Routing is the foundational mechanism in FastAPI that determines how an application responds to client requests sent to specific endpoints (URIs) via HTTP methods.

1.1 What is a Route in FastAPI?

In FastAPI, a **route** consists of an HTTP verb (such as GET, POST, PUT, DELETE), a URL path pattern, and an asynchronous or synchronous path operation function that executes when the path matches the request.

Key Features of FastAPI Routing:

- **Automatic OpenAPI Documentation:** Every defined route automatically generates Swagger UI and ReDoc endpoints.
- **Type Validation:** Path and query parameters are automatically converted and validated using Pydantic.
- **Asynchronous Support:** Native support for Python's `async` and `await` keywords for asynchronous I/O.
- **Performance:** Built on Starlette and Pydantic, making it one of the fastest Python web frameworks available.

1.2 HTTP Methods Table

HTTP Decorator	CRUD Action	Primary Usage
`@app.get()`	Read	Retrieve resources or data lists from server
`@app.post()`	Create	Submit new entities or payload to the server
`@app.put()`	Update (Full)	Replace an existing resource completely
`@app.patch()`	Update (Partial)	Modify specific fields of an existing resource
`@app.delete()`	Delete	Remove a specified resource from storage

2. Basic Route Creation & Decorators

Creating routes in FastAPI begins with instantiating the `FastAPI` application class and decorating Python functions with path operation decorators.

2.1 Basic Setup Code Example

The simplest FastAPI application defines path operations using decorators on the main app instance.

```
from fastapi import FastAPI

app = FastAPI(
    title="Smart Project API",
    description="Routing Architecture for Smart Applications",
    version="1.0.0"
)

# Root route definition
@app.get("/")
async def read_root():
    return {"message": "Welcome to Smart Project API"}

# Health check endpoint
@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "active"}
```

2.2 Path Operation Decorator Parameters

Decorators accept metadata parameters that enhance API documentation and structure:

```
@app.get(
    "/items/{item_id}",
    summary="Fetch single item",
    description="Retrieves a specific item record by its unique identifier.",
    tags=["Items"],
    response_description="The requested item payload"
)
async def get_item(item_id: int):
    return {"item_id": item_id, "name": "Sample Item"}
```

3. Path Parameters & Data Types

Path parameters allow you to capture dynamic values from the URL path. FastAPI enforces strict type declaration using Python standard type annotations.

3.1 Declaring Path Parameters with Types

When a parameter is declared in the function arguments matching a placeholder in the route path, FastAPI automatically parses, type-casts, and validates it.

```
from fastapi import FastAPI, Path

app = FastAPI()

# Integer Path Parameter with Validation
@app.get("/users/{user_id}")
async def get_user_by_id(
    user_id: int = Path(..., title="User ID", ge=1, description="Must be >= 1")
):
    return {"user_id": user_id, "user_type": "standard"}

# String Path Parameter with Enum Constraint
from enum import Enum

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep Learning FTW!"}
    return {"model_name": model_name, "message": "LeNet or ResNet selected"}
```

Automatic Data Validation:

If a user sends a request to `/users/abc`, FastAPI returns an automatic `422 Unprocessable Entity` response containing detailed JSON error details, preventing invalid data processing.

4. Query Parameters & Filtering

Function parameters that are not part of the route path are automatically interpreted as **Query Parameters** (URL parameters following the ? mark).

4.1 Handling Optional & Required Query Parameters

```
from typing import Optional
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/products/")
async def list_products(
    page: int = Query(1, ge=1, description="Page number"),
    limit: int = Query(10, le=100, description="Items per page"),
    search: Optional[str] = Query(None, min_length=3, max_length=50),
    is_active: bool = True
):
    return {
        "page": page,
        "limit": limit,
        "search": search,
        "is_active": is_active,
        "results": []
    }
```

4.2 Comparison Table: Path vs Query Parameters

Feature	Path Parameters	Query Parameters
URL Syntax	`/users/123`	`/users?page=1&limit=10`
Primary Purpose	Identify a specific resource entity	Filter, sort, paginate, or search resources
Mandatory Status	Always required	Can be optional with defaults

5. Request Body & Pydantic Schema Routing

To receive JSON payloads from HTTP POST, PUT, or PATCH requests, FastAPI leverages Pydantic models to define strict schema contracts.

5.1 Defining Input Models for Routes

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional

class UserRegisterSchema(BaseModel):
    username: str = Field(..., min_length=3, max_length=30)
    email: EmailStr
    password: str = Field(..., min_length=8)
    age: Optional[int] = Field(None, ge=18, le=120)

@app.post("/api/v1/auth/register", status_code=201)
async def register_user(user_data: UserRegisterSchema):
    # Payload is automatically converted to Pydantic object
    return {
        "status": "success",
        "username": user_data.username,
        "email": user_data.email
    }
```

Key Advantages:

- Automatic JSON parsing from HTTP request body.
- Validation against defined types and field bounds.
- Full IDE auto-completion and static linting support.

6. Modular Routing with APIRouter

In production applications, keeping all routes in a single file leads to unmaintainable code. FastAPI provides `APIRouter` to group and split routes into modular, feature-based files.

6.1 Defining a Router Module (e.g., `routes/auth.py`)

```
from fastapi import APIRouter

router = APIRouter(
    prefix="/auth",
    tags=["Authentication"],
    responses={404: {"description": "Not found"}}
)

@router.post("/login")
async def login():
    return {"message": "Login successful"}

@router.post("/logout")
async def logout():
    return {"message": "Logged out successfully"}
```

6.2 Including Routers in Main App (`main.py`)

```
from fastapi import FastAPI
from routes import auth, users, products

app = FastAPI(title="Modular Smart Project")

# Registering modular routes
app.include_router(auth.router, prefix="/api/v1")
app.include_router(users.router, prefix="/api/v1")
app.include_router(products.router, prefix="/api/v1")
```

7. Route Security & Dependency Injection

FastAPI includes a powerful **Dependency Injection** system (`Depends`) that enables route protection, authentication checks, and database session injection.

7.1 Implementing Route Guards for Protected Endpoints

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

app = FastAPI()
security = HTTPBearer()

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)):
    token = credentials.credentials
    if token != "secret-smart-token":
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired access token"
        )
    return {"user_id": 42, "role": "admin"}

# Protected Route enforcing token validation
@app.get("/api/v1/dashboard")
async def get_dashboard(current_user: dict = Depends(verify_token)):
    return {
        "message": "Welcome to protected dashboard",
        "user": current_user
    }
```

8. Error Handling & Response Routing

Graceful exception handling and standardized HTTP responses are vital for reliable production routes.

8.1 Custom Response Status Codes and Exceptions

```
from fastapi import FastAPI, HTTPException, status
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id == 0:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Item ID 0 is invalid."
        )
    if item_id > 100:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Item not found in database."
        )
    return {"item_id": item_id, "name": "Valid Item"}
```

Standard HTTP Response Statuses:

- **200 OK:** Standard success response.
- **201 Created:** Successful creation via POST.
- **400 Bad Request:** Invalid client payload input.
- **401 Unauthorized:** Missing or invalid auth token.
- **404 Not Found:** Resource requested does not exist.

9. Best Practices & Routing Checklist

Adhering to standard API conventions ensures scalable, clean, and maintainable routing across teams.

9.1 FastAPI Routing Best Practices

1. **Use Modular Routers:** Divide endpoints by domain into separate `APIRouter` files.
2. **Enforce Type Annotations:** Explicitly declare input/output models using Pydantic schemas.
3. **API Versioning:** Prefix routes with explicit versions (e.g., `/api/v1/`).
4. **Leverage Asynchronous Operators:** Use `async def` for non-blocking I/O operations (DB queries, external API calls).
5. **Centralize Dependency Guards:** Pass shared logic like token checks via `Depends()`.

9.2 Routing Architecture Summary

Component	Purpose	FastAPI Class / Decorator
App Core	Central server entry point	`FastAPI()`
Sub-Router	Feature boundary modularization	`APIRouter()`
Payload Model	Request body validation	`pydantic.BaseModel`
Route Guard	Authentication / Middleware injection	`Depends()`

> **"FastAPI routing combines Python type hints with high performance to build scalable, documented APIs effortlessly."**